

Serindit Pay — Merchant Payment Orchestration Platform

Detailed Design

Architecture, requirements, and validation criteria for build

Version	1.0
Status	Design phase — build not started
Last updated	2026-09-14 (architecture) · consolidated 2026-09-21
Prepared by	Serindit Pay Platform Engineering — Payments Core Team
Companion documents	MPOP Conceptual Design v1.2; PCI DSS Scoping Memo (draft); Acquirer Connectivity Matrix

Table of Contents

- 1. Purpose and Scope
- 2. Requirements
- 3. Foundational Principles
- 4. Markets and Payment Method Coverage
- 5. Target Architecture
- 6. Merchant and Account Model
- 7. Payment Lifecycle and State Machine
- 8. Authorization Flow
- 9. Idempotency
- 10. Routing Engine
- 11. Retries and Cascading
- 12. Card Vault and PCI DSS Scoping
- 13. Fraud Scoring Hook
- 14. Ledger and Double-Entry Accounting
- 15. Amounts, Currencies and FX
- 16. Reconciliation, Settlement and Payouts
- 17. Merchant Webhooks
- 18. Merchant Admin Plane
- 19. Data Model
- 20. Resilience, Availability and Disaster Recovery
- 21. Latency and Capacity Budget
- 22. Observability and Audit
- 23. Prior Art and Reference Architecture
- 24. Confirmed Decisions
- 25. Pending Backlog
- 26. Validation and Acceptance Criteria
- 27. Implementation Readiness Assessment
- 28. Build Phases

1. Purpose and Scope

This document describes the architecture of the Serindit Pay Merchant Payment Orchestration Platform (MPOP) — the system that accepts payment requests from Serindit Pay's merchants, routes each one to an appropriate acquirer, e-wallet provider, or bank-transfer rail, and accounts for every resulting movement of money until it is settled to the merchant. It is written to a level of technical detail sufficient for an engineering team to begin implementation. Section 27 gives an explicit assessment of where that is and is not yet true.

MPOP replaces five per-market integration stacks that share no routing, no ledger, and reconcile by spreadsheet.

Checkout UIs, hosted pages and mobile SDKs consume MPOP's API and are specified separately.

MPOP provides one merchant API across five markets; rule-based routing with cascading on retryable failures; idempotent handling of every mutating call; a tokenising card vault that confines PCI DSS scope to a small enclave; a synchronous fraud-scoring hook; a double-entry ledger; daily reconciliation against settlement files; signed merchant webhooks; and a merchant admin plane.

In scope

Payment acceptance, routing, retries, idempotency, vaulting, fraud-scoring integration, ledger, reconciliation, merchant payouts, merchant webhooks, and the merchant admin plane, for approximately 15,000 active merchants with a peak of 2,000 payment-creation transactions per second (TPS).

Out of scope

- Dispute and chargeback case management (disputes are ingested as ledger events only; see Pending Backlog).
- Merchant onboarding and KYB (handled by the Merchant Risk platform; MPOP consumes an approved-merchant event).
- 3-D Secure server (a third-party 3DS server is integrated as a step in the authorization flow; its internals are not specified here).
- Lending, buy-now-pay-later, and instalment products.

Volume baseline

Metric	2025 actual	Design target (2027)
Active merchants	9,800	15,000
Payment attempts per day (average)	5.1 M	10.5 M
Average TPS	59	122
Peak TPS (campaign events: 9.9, 11.11, 12.12)	1,140	2,000
Card share of attempts (e-wallet ~30%, bank transfer ~15%)	54%	55%
Largest single merchant share at peak	41%	45%

2. Requirements

Each requirement carries an ID used again in Section 26 (Validation and Acceptance Criteria) and Section 27 (Implementation Readiness Assessment).

2.1 Functional Requirements

ID	Requirement
FR-1	The platform shall expose one versioned merchant API supporting create payment, authorize, capture (full and partial), void, refund (full and partial), and retrieve, for all payment methods listed in Section 4.
FR-2	Every payment shall be in exactly one state of the state machine in Section 7 at any time; only the transitions listed there shall be permitted, and every transition shall be persisted before it is acknowledged to the merchant.
FR-3	The platform shall support the payment methods per market listed in Section 4, each through an adapter that implements the common Connector interface.
FR-4	Create payment, capture, void and refund requests shall require an Idempotency-Key header. Retrieve requests shall not.
FR-5	Requests carrying the same merchant and Idempotency-Key shall produce at most one authorization, capture, void or refund at the downstream provider, and shall return the identical response, including when duplicate requests arrive concurrently. A reused key with a different request body shall be rejected with HTTP 422.
FR-6	The Routing Engine shall select, for each payment attempt, an acquirer or provider from the eligible set determined by payment method, market, currency, card scheme, BIN country, and the merchant's contracted acquirers.
FR-7	On a retryable outcome (Section 11), the Retry Engine shall resubmit a card payment to the next eligible acquirer, up to two additional attempts, and shall never reattempt where the card scheme's rules prohibit it.
FR-8	For each transaction, the Routing Engine shall prefer the lowest-cost eligible acquirer unless doing so reduces the expected

ID	Requirement
	authorization rate by more than 2%.
FR-9	Every card payment and every e-wallet payment above the merchant's configured threshold shall be scored by the fraud-scoring hook before authorization, producing ACCEPT, REVIEW or REJECT.
FR-10	Every movement of money (authorization capture, refund, fee, FX conversion, settlement, payout, dispute debit) shall be recorded as a balanced double-entry journal in the ledger.
FR-11	The platform shall reconcile, daily, every acquirer, e-wallet and bank settlement line against the internal payment record and the ledger (three-way match), and shall raise an exception for every unmatched or mismatched line.
FR-12	The platform shall deliver a signed webhook to the merchant for every payment, refund and payout state change.
FR-13	The merchant admin plane shall let merchant users manage users and roles, API keys, webhook endpoints, refunds, reports, and payout bank account details.
FR-14	Card-on-file and recurring payments shall use a stored credential with the scheme's stored-credential (MIT/CIT) indicators, and shall use network tokens where the issuer supports them.
FR-15	Refunds shall be supported for every payment method; where a method has no refund API, the refund shall be executed as a bank payout to the customer's nominated account.
FR-16	Every merchant API mutation and every admin-plane action shall produce an audit record (Section 22).
FR-17	The platform shall pay out merchant balances to the merchant's registered bank account on the merchant's contracted schedule (T+1, T+2 or weekly).

2.2 Non-Functional Requirements

ID	Requirement
NFR-1	The platform shall sustain 2,000 create-payment TPS for four continuous hours with no degradation of NFR-2, across all merchants concurrently.
NFR-2	End-to-end p99 latency for a card authorization, measured at the API edge and including any cascade, shall not exceed 1,500 ms. Orchestration overhead (all time not spent waiting on an acquirer) shall not exceed 250 ms at p99.
NFR-3	Monthly availability of the merchant payment API shall be at least 99.95%.
NFR-4	Recovery point objective for authorized payments and ledger postings shall be zero, including on loss of an entire AWS region. Recovery time objective shall be 30 minutes.
NFR-5	The platform shall be assessed as a PCI DSS v4.0 Level 1 service provider. The cardholder data environment (CDE) shall be limited to the components listed as in-scope in Section 12.2.
NFR-6	Sensitive authentication data shall be handled per PCI DSS v4.0 Requirements 3.2.1 and 3.3.2: stored only in encrypted form, in the Vault, and deleted at settlement of the associated payment.
NFR-7	Personal data shall be processed in accordance with the personal data protection laws of each market of operation (Singapore PDPA 2012, Malaysia PDPA 2010, Indonesia Law 27/2022, Thailand PDPA 2019, Philippines DPA 2012), including purpose limitation, retention limits, and access logging.
NFR-8	At least 99.9% of settlement lines shall be auto-matched; merchant settlement reports for business day T shall be published by 08:00 SGT on T+1.
NFR-9	Per-merchant rate limits shall prevent any single merchant from degrading latency or availability for others.
NFR-10	A new acquirer or provider shall be onboarded by implementing the Connector interface and adding configuration, with no changes to the Payments API, Routing Engine, Retry Engine, or Ledger.
NFR-11	Every payment shall be traceable end-to-end (merchant request, routing decision, each attempt, ledger journals, webhooks, settlement line) by payment_id within 5 seconds of each event.
NFR-12	Audit records shall be immutable and retained for five years.

3. Foundational Principles

These ten principles govern every design decision in the platform. Where a later section appears to conflict with one of these, the principle wins.

#	Principle
P1	One payment, one truth. The payment record and the ledger are the source of truth. Acquirer, wallet and bank responses are inputs to it, never a substitute for it.
P2	Card data stays in the Vault. Primary account numbers (PAN) and sensitive authentication data never leave the Vault boundary

#	Principle
	except over the Card Adapter's connection to an acquirer. All other services handle tokens only.
P3	Idempotency is scoped to (merchant, key). Every mutating call is deduplicated on the pair of merchant identity and client-supplied key.
P4	Money is integers. Amounts are stored and transmitted as integer minor units with an explicit ISO 4217 currency code. No floating-point type ever holds an amount.
P5	The ledger is append-only. Corrections are new journals that reverse or adjust; nothing in the ledger is updated or deleted.
P6	Adapters absorb difference. Every acquirer, wallet and rail quirk lives in its adapter. The core never branches on provider identity.
P7	At-least-once delivery, exactly-once effect. Messages may be delivered more than once; consumers deduplicate on a stable identifier.
P8	Fail closed on security, degrade gracefully on enrichment. Authentication and authorization failures deny. Optional enrichment (fraud signals, BIN metadata) degrades to documented defaults.
P9	Least privilege and separation of duties. Every human and service identity has only the permissions its role requires; money-moving permissions are separated from configuration permissions.
P10	Everything is auditable. Every mutation, routing decision, and admin action can be reconstructed after the fact.

4. Markets and Payment Method Coverage

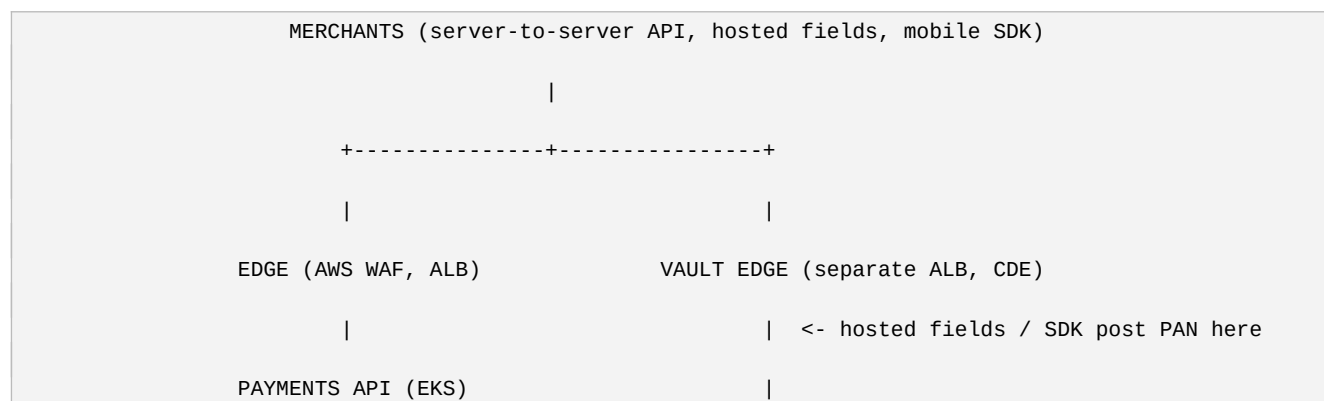
Market	Merchants (2027 target)	Cards	E-wallets	Bank transfer
Singapore	3,100	Visa, Mastercard, Amex (ACQ-SG1, ACQ-SG2)	GrabPay, ShopeePay (via WAG-1)	PayNow QR
Malaysia	3,000	Visa, Mastercard (ACQ-MY1, ACQ-SG2)	Touch 'n Go eWallet, GrabPay, Boost (via WAG-1)	FPX, DuitNow QR
Indonesia	4,200	Visa, Mastercard (ACQ-ID1, ACQ-SG2)	OVO, DANA, ShopeePay (via WAG-1)	QRIS, bank virtual accounts
Thailand	2,300	Visa, Mastercard (ACQ-TH1, ACQ-SG2)	TrueMoney, ShopeePay (via WAG-1)	PromptPay QR
Philippines	2,400	Visa, Mastercard (ACQ-PH1, ACQ-SG2)	GCash, Maya (via WAG-1)	InstaPay QR

ACQ-SG2 is a regional acquirer licensed in all five markets and the default cascade secondary. WAG-1 is a wallet aggregator fronting all listed e-wallets. Bank rails are reached through per-market sponsor banks.

E-wallet and bank-transfer payments are customer-redirect or QR flows: the customer completes payment in the wallet or banking app, and the provider notifies MPOP asynchronously. They are not cascaded (Section 11).

5. Target Architecture

Every payment, from every merchant, flows through the same governed path. No service other than an adapter speaks to an external provider, and no service other than the Vault holds card data.



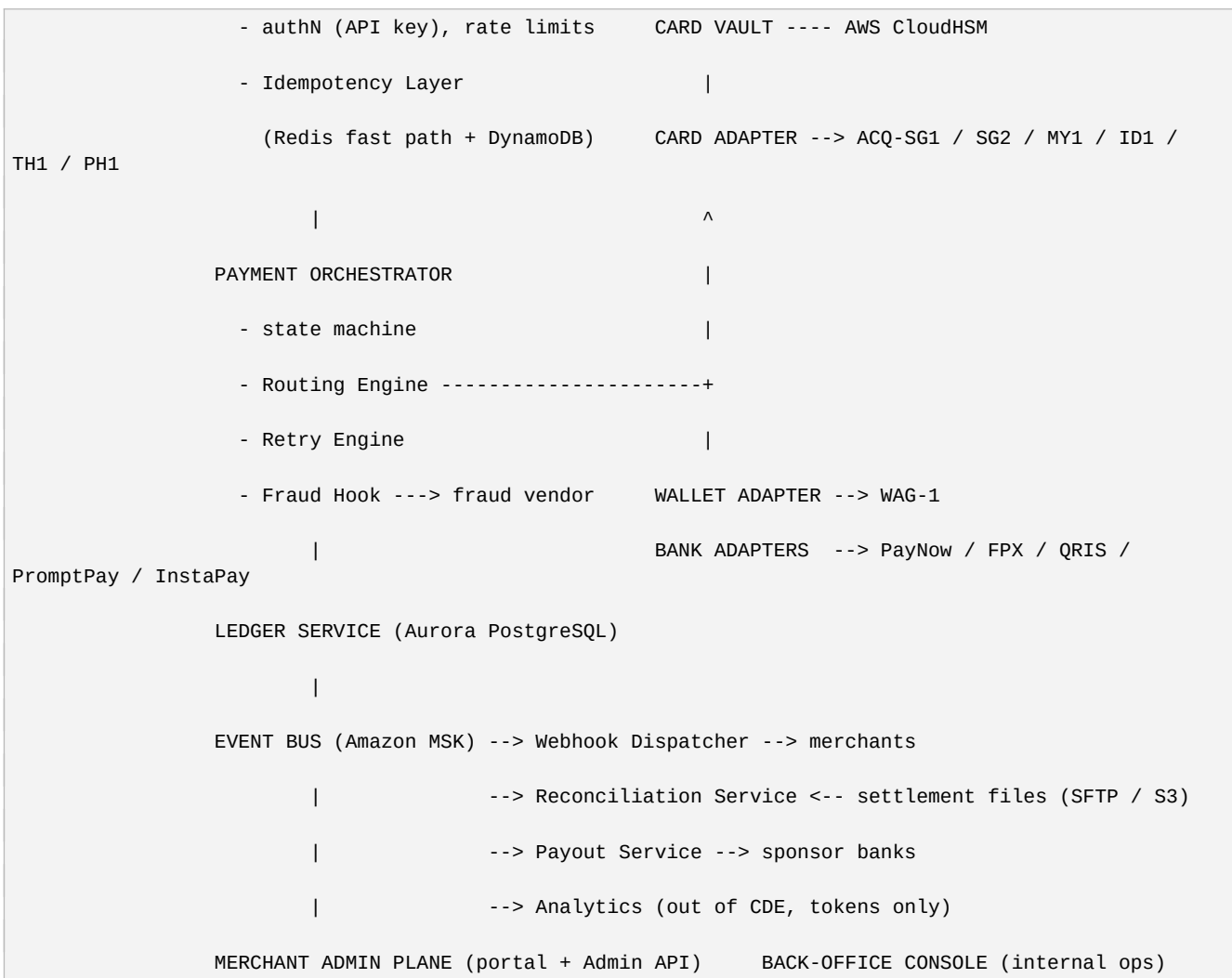


Figure 1 — Target architecture. The Vault, CloudHSM, Card Adapter and Vault Edge form the CDE; everything else handles tokens.

Layer responsibilities

- **Edge** — AWS WAF, TLS termination, request size limits.
- **Payments API** — merchant authentication by secret API key, per-merchant rate limits (token bucket, configurable per merchant tier), request validation, idempotency (Section 9).
- **Payment Orchestrator** — owns the payment state machine (Section 7), invokes the Fraud Hook (Section 13), the Routing Engine (Section 10) and the Retry Engine (Section 11), and writes state transitions and ledger journals in one database transaction with a transactional outbox.
- **Card Vault** — tokenises PAN on capture from hosted fields or the server-to-server card endpoint; stores encrypted card data; detokenises only for the Card Adapter (Section 12).
- **Adapters** — one per provider family. Translate the common Connector interface to provider protocols, map response codes into MPOP's outcome classes, and normalise amounts (Section 15).
- **Ledger Service** — double-entry journals and balances (Section 14).
- **Event Bus** — Amazon MSK (Kafka). Topics per domain event; outbox relay publishes committed events.
- **Webhook Dispatcher, Reconciliation, Payouts, Admin Plane** — Sections 16–18.

All workloads run on Amazon EKS in ap-southeast-1 (Singapore) across three Availability Zones. The CDE runs in a dedicated AWS account and VPC with its own EKS cluster, connected to the core VPC only through a private endpoint exposing the Vault's tokenise, detokenise-for-adapter and card-metadata operations.

6. Merchant and Account Model

Entity	Description	Key fields
Merchant	A legal entity contracted with Serindit Pay.	merchant_id, legal name, home market, risk tier (LOW / MEDIUM / HIGH), settlement schedule
Sub-merchant	A seller under a marketplace merchant. Payments may be attributed to a sub-merchant for split settlement.	sub_merchant_id, parent merchant_id, payout account
Merchant account	A (merchant, market, currency) combination with its own acquirer contracts and pricing.	merchant_account_id, market, currency, contracted acquirers, MCC
API key	Secret key (server-side) or publishable key (client-side, tokenise-only).	key_id, prefix, SHA-256 hash of secret, scopes, created_by, last_used_at
Webhook endpoint	HTTPS URL plus signing secret.	endpoint_id, URL, event filter, signing secret (encrypted), status
Payout account	Bank account for merchant payouts.	bank code, account number (encrypted), account name, verified_at

Secret API keys are displayed once and stored only as a SHA-256 hash with a visible prefix. Publishable keys can call only tokenise and client-side confirmation endpoints.

7. Payment Lifecycle and State Machine

Every payment is a payment record with one current state and an append-only list of payment_attempt records (one per acquirer/provider submission).

7.1 States

State	Meaning
CREATED	Accepted and validated; no provider contacted.
REQUIRES_ACTION	Awaiting customer action (3DS challenge, wallet redirect, QR scan).
AUTHORISING	At least one attempt is in flight with a provider.
AUTHORISED	Funds reserved (cards) — capture pending.
CAPTURED	Fully or partially captured; awaiting settlement.
SUCCEEDED	Terminal success for wallet and bank-transfer payments (no separate capture).
SETTLED	Settlement line matched in reconciliation.
FAILED	Terminal failure; no funds moved.
CANCELLED	Voided before capture, or expired without customer action.
REFUNDED / PARTIALLY_REFUNDED	One or more refunds succeeded against a captured or succeeded payment.

7.2 Permitted transitions

CREATED	-> REQUIRES_ACTION AUTHORISING FAILED CANCELLED
REQUIRES_ACTION	-> AUTHORISING CANCELLED (expiry)
AUTHORISING	-> AUTHORISED SUCCEEDED FAILED
AUTHORISED	-> CAPTURED CANCELLED (void or auth expiry)
CAPTURED	-> SETTLED PARTIALLY_REFUNDED REFUNDED
SUCCEEDED	-> SETTLED PARTIALLY_REFUNDED REFUNDED
SETTLED	-> PARTIALLY_REFUNDED REFUNDED
PARTIALLY_REFUNDED	-> REFUNDED

Each transition is written with an optimistic-concurrency version check (UPDATE payment SET state = \$new, version =

version + 1 WHERE payment_id = \$id AND version = \$expected). A failed version check aborts the transaction and the caller re-reads. A transition and its ledger journal (where one applies) and its outbox event are committed in one PostgreSQL transaction.

An expiry sweeper voids uncaptured card authorizations one day before their scheme expiry.

8. Authorization Flow

The card authorization flow, end to end:

```
1. Merchant -> Payments API: POST /v1/payments (Idempotency-Key, payment_method_token, amount,
currency)

2. Payments API: authenticate key; rate-limit check; schema validation

3. Idempotency Layer (Section 9): fast-path lookup; acquire lock or return stored response

4. Orchestrator: create payment (CREATED); outbox event payment.created

5. Fraud Hook (Section 13): score -> ACCEPT | REVIEW | REJECT (hard timeout 150 ms)

    REJECT -> FAILED (reason fraud_rejected); respond

    REVIEW -> per merchant config: authorize-and-hold-capture, or FAILED

6. 3DS (if required by merchant rule, issuer mandate, or market regulation): REQUIRES_ACTION; resume
on challenge result

7. Routing Engine (Section 10): ordered candidate list [primary, secondary, tertiary]

8. Orchestrator -> AUTHORISING; Card Adapter submits attempt 1

9. Outcome classification (Section 11): approved | final decline | retryable

    retryable -> Retry Engine submits next candidate (up to 2 more attempts)

10. Orchestrator -> AUTHORISED or FAILED; ledger journal (authorization memo); outbox event

11. Idempotency Layer: store final response; release lock

12. Payments API -> Merchant: 200/201 with payment object

13. Webhook Dispatcher: payment.authorised / payment.failed (asynchronous)
```

E-wallet and bank-transfer flows diverge at step 7: the adapter returns a redirect URL or QR payload, the payment moves to REQUIRES_ACTION, and the provider's signed asynchronous notification, deduplicated on the provider's reference, drives the final transition.

9. Idempotency

9.1 Contract

- Idempotency-Key is a client-generated string of 1–255 characters. We recommend a UUIDv4; merchants frequently use their own order or invoice identifiers, which is accepted.
- The key is bound to a hash (SHA-256) of the canonicalised request body. Reuse with a different body returns HTTP 422 idempotency_key_reused.
- Records are retained for 24 hours from first use. A request after expiry is treated as new.
- While the first request is in flight, a duplicate receives HTTP 409 request_in_progress with Retry-After: 1.

9.2 Two-tier implementation

Fast path (Redis). Amazon ElastiCache for Redis (cluster mode enabled, three shards, one replica each). On completion of a request, the final response is written to `idem:resp:{idempotency_key}` with a 24-hour TTL. On every incoming mutating request, the Payments API checks this key first; on a hit, it returns the stored response immediately without touching DynamoDB or the Orchestrator. The fast path absorbs client retry storms during acquirer brownouts, when merchants' HTTP clients retry aggressively.

Durable path (DynamoDB). Table `payments-idempotency`:

Attribute	Type	Notes
<code>merchant_id</code>	String	Partition key
<code>idempotency_key</code>	String	Sort key
<code>request_hash</code>	String	SHA-256 of canonical body
<code>status</code>	String	IN_PROGRESS / COMPLETED
<code>payment_id</code>	String	Set once known
<code>response_status</code>	Number	HTTP status
<code>response_body</code>	String	Serialised response, up to 4 KB
<code>created_at</code>	String	ISO 8601; LSI sort key <code>lsi_created_at</code>
<code>ttl</code>	Number	Epoch seconds, <code>created_at</code> + 24 h

On a fast-path miss, the Payments API performs a conditional `PutItem` with `attribute_not_exists(idempotency_key)` and status `IN_PROGRESS`. If the put succeeds, the request proceeds. If it fails, the API reads the item: `COMPLETED` with matching hash returns the stored response; `COMPLETED` with a different hash returns 422; `IN_PROGRESS` returns 409. On completion, an `updateItem` sets `COMPLETED`, `response_status` and `response_body`, and the response is written to the Redis fast path.

A lock left `IN_PROGRESS` by a crashed API pod is recovered by the Orchestrator's stuck-payment sweeper (Section 20.3), which resolves the payment and completes the record.

9.3 Capacity

The table uses on-demand capacity mode. DynamoDB serves each partition key value from a single partition, and a single partition sustains up to 10,000 write capacity units per second. Our largest merchant (marketplace M-0001) peaks at approximately 900 create-payment TPS during campaign events (45% of 2,000 TPS). Each request consumes two writes — lock acquisition and completion update — or about 1,800 WCU at peak, comfortably within the per-partition ceiling. Keying by `merchant_id` keeps each merchant's records co-located, which the local secondary index on `created_at` uses to serve the back-office "recent requests by merchant" view without a scan.

10. Routing Engine

10.1 Eligibility

For each card payment, the Routing Engine computes the eligible acquirer set as the intersection of:

1. Acquirers contracted for the merchant account (market, currency).
2. Acquirers supporting the card scheme and, where relevant, the card's BIN country (some acquirers decline cross-border BINs or price them prohibitively).
3. Acquirers currently healthy (circuit breaker closed; see 10.4).
4. Merchant-level overrides (pin to acquirer, exclude acquirer).

10.2 Ranking

Eligible acquirers are ranked by a weighted score:

```
score(a) = w_approval * approval_prior(a, scheme, bin_country, mcc) - w_cost * normalised_cost(a, txn) - w_latency * latency_p95(a)    # highest score first
```

`normalised_cost` uses the contracted MDR, interchange-plus components where applicable, and cross-border surcharges. `approval_prior` is the trailing 28-day approval rate for the (acquirer, scheme, BIN country, MCC) cell,

falling back to coarser cells when the sample is under 500 attempts. Weights are configured per merchant account; the default emphasises cost.

10.3 Cost–approval trade-off

The cost-preferred ranking is subject to FR-8. At routing time, the engine compares the approval prior of the lowest-cost acquirer against that of the highest-approval acquirer and applies the cost preference when the difference is within the configured tolerance (default 2). Separately, the weekly Routing Review compares each merchant's realised authorization rate under cost-preferred routing against a control slice (5% of traffic routed approval-first) and adjusts weights where the merchant's authorization rate has fallen by more than the tolerance.

10.4 Health and circuit breaking

Each acquirer connection has a circuit breaker (30-second window; opens above 20% errors with at least 50 calls; half-opens after 15 seconds). An open breaker removes the acquirer from eligibility.

10.5 Baseline outcomes

In the 2025 baseline, across all markets, card attempts resolved as: approved 88.1%, final (hard) declines 8.1%, and retryable outcomes — soft declines eligible for reattempt plus acquirer-side errors and timeouts — 3.8%. Cascading recovered approximately 31% of retryable outcomes in a three-month pilot on ACQ-SG1 → ACQ-SG2.

11. Retries and Cascading

11.1 Outcome classification

Adapters map provider responses into three outcome classes:

Class	Examples	Action
Approved	ISO 8583 response 00; provider "authorised"	Proceed to AUTHORISED
Final	Hard declines (14 invalid card, 41/43 lost/stolen, 54 expired, 51 insufficient funds, 57 not permitted); Visa Category 1 responses; Mastercard Merchant Advice Codes 03 and 21	FAILED; never reattempted on any acquirer
Retryable	Soft declines 05 (do not honour) and 91 (issuer unavailable) and 96 (system malfunction); acquirer HTTP 5xx; connection refused; no response within 2,500 ms	Cascade to next candidate

11.2 Cascade procedure

On a retryable outcome, the Retry Engine immediately submits a new attempt, with a new `attempt_id`, to the next acquirer in the Routing Engine's candidate list. At most two additional attempts are made (three in total). Each attempt is persisted before submission. Because each attempt carries its own `attempt_id` and the previous attempt did not succeed, cascading cannot create a duplicate charge: the ledger records only the approved attempt. Responses that arrive for an attempt after its 2,500 ms timeout are logged against the attempt and discarded.

For card payments, each cascade attempt re-presents the full card credential, which the Card Adapter retrieves from the Vault (including the card verification code; see Section 12.4) so that the second acquirer receives the same data as the first.

11.3 Scheme compliance

The Retry Engine maintains a per-card (PAN fingerprint) and per-merchant reattempt counter over a rolling 30-day window and stops reattempting at the scheme limits applicable to the card. Responses carrying "do not reattempt" advice are classed as Final regardless of response code.

11.4 Non-card methods

E-wallet and bank-transfer payments are never cascaded: the customer has already been directed to a specific wallet or bank. A failure returns FAILED and the merchant may create a new payment.

12. Card Vault and PCI DSS Scoping

12.1 Vault design

- **Tokenise.** Hosted fields, the mobile SDK, and the server-to-server card endpoint post PAN, expiry and CVC directly to the Vault Edge. The Vault returns a `card_token` (format-independent, random 24 characters, prefixed `ctk_`) plus non-sensitive metadata: BIN (first 8), last 4, scheme, funding type, issuer country.
- **Storage.** Card records are encrypted with envelope encryption: each record has its own data encryption key (DEK, AES-256-GCM), wrapped by a key-encryption key (KEK) held in AWS CloudHSM.
- **Detokenise.** Only the Card Adapter's service identity may call `detokenise`. Each call unwraps the record's DEK inside the HSM; DEKs are not cached, so plaintext key material never persists in application memory.
- **PAN fingerprint.** A keyed HMAC-SHA-256 of the PAN (key in the HSM) supports duplicate-card detection and the counters in Section 11.3.

12.2 CDE scope

Component	PCI DSS scope	Basis
Vault Edge (ALB, WAF)	CDE	Receives PAN and CVC
Card Vault	CDE	Stores, processes PAN
AWS CloudHSM	CDE	Holds KEKs
Card Adapter	CDE	Transmits PAN to acquirers
Payments API, Orchestrator, Routing, Retry	Out of scope	Token, BIN, last 4 only
Fraud Hook	Out of scope	Token, BIN, last 4 only
Ledger, Reconciliation, Payout, Webhooks, Admin Plane	Out of scope	No card data
Analytics	Out of scope	Token, BIN, last 4 only

Segmentation is enforced by a dedicated VPC, security groups allowing only the private endpoint, and IAM boundaries, and is penetration-tested every six months.

12.3 HSM topology

The Vault's KEKs are held in an AWS CloudHSM cluster with one HSM in `ap-southeast-1a`. CloudHSM takes automatic daily backups of the cluster to S3, from which a new HSM can be created in any AZ. A second HSM will be added to the cluster when sustained card volume exceeds 1,500 TPS. If the HSM is unreachable, the Vault returns HTTP 503 and the Card Adapter maps this to a retryable outcome.

12.4 Sensitive authentication data

The CVC is required for the initial customer-initiated authorization and, at several acquirers, for each re-presentation of the credential during a cascade and for incremental authorizations (ride-hailing tips, hotel extensions). The Vault therefore stores the CVC encrypted under a separate KEK, and deletes it when the Reconciliation Service marks the associated payment `SETTLED`, or after 72 hours, whichever is first. This is permitted by PCI DSS v4.0 Requirements 3.2.1 and 3.3.2, which allow sensitive authentication data to be retained in encrypted form until the transaction is settled. Track data and PIN blocks are never accepted.

12.5 Network tokens

For card-on-file and recurring payments, the Vault provisions a Visa Token Service (VTS) or Mastercard MDES network token against each stored card and uses it, with its cryptogram, for all subsequent merchant-initiated and customer-initiated card-on-file authorizations. The PAN is used only where the issuer does not support network tokens. Network tokens are refreshed automatically by lifecycle notifications on card reissue, reducing declines from expired credentials.

13. Fraud Scoring Hook

MPOP integrates a third-party fraud-scoring vendor (FRV-1) behind a Fraud Hook service. The hook is called synchronously at step 5 of the authorization flow.

13.1 Request payload

Field	Source	Purpose
payment_id, merchant_id, MCC	Orchestrator	Context
amount, currency	Orchestrator	Amount-based rules
card_token, BIN (first 8), last 4, issuer country	Vault metadata	Card attributes
card_number	Vault detokenise	Cross-merchant velocity graph
device fingerprint, IP address, user agent	Merchant SDK	Device signals
customer email (SHA-256), phone (SHA-256)	Merchant request	Identity linkage
shipping and billing country	Merchant request	Geo mismatch rules

FRV-1's consortium velocity graph is keyed on the full card number, which allows it to link the same card across its other clients in the region; scoring quality falls materially without it. The Fraud Hook obtains the PAN through the Vault's detokenise operation, using the Card Adapter client library and service role.

13.2 Decisions and fallback

- Hard timeout 150 ms. On timeout or vendor error: LOW and MEDIUM risk-tier merchants receive ACCEPT; HIGH risk-tier merchants receive REVIEW.
- Thresholds (score → decision) are configured per merchant account in the admin plane by Serindit Pay's risk team, not by the merchant.
- Every decision, score and FRV-1 model version is stored on the payment.

14. Ledger and Double-Entry Accounting

14.1 Model

The ledger is a set of accounts and an append-only table of journals, each consisting of two or more postings. A journal is valid only if, for each currency, the sum of debit postings equals the sum of credit postings. This invariant is enforced by a deferred constraint trigger at commit and re-verified nightly by a full recomputation.

Account	Type	Per
acquirer_receivable	Asset	acquirer, currency
settlement_cash	Asset	sponsor bank account, currency
merchant_payable	Liability	merchant account, currency
refund_clearing	Liability	currency
fee_revenue	Revenue	currency
scheme_and_acquirer_fees	Expense	acquirer, currency
fx_position	Asset/Liability	currency pair
dispute_receivable	Asset	merchant account, currency

14.2 Example journals (SGD 100.00, MDR 2.80%, acquirer cost 1.90%)

Capture:	Dr acquirer_receivable[ACQ-SG1, SGD]	10000	
	Cr merchant_payable[M-1234, SGD]		9720
	Cr fee_revenue[SGD]		280
Settlement:	Dr settlement_cash[DBS-OPS, SGD]	9810	
	Dr scheme_and_acquirer_fees[ACQ-SG1, SGD]	190	
	Cr acquirer_receivable[ACQ-SG1, SGD]		10000

Payout:	Dr merchant_payable[M-1234, SGD]	9720
	Cr settlement_cash[DBS-OPS, SGD]	9720

14.3 Rules

- Postings are never updated or deleted. A correction is a new journal referencing the one it corrects, with a reason code and approver.
- Journals are written in the same PostgreSQL transaction as the payment state transition that causes them, together with an outbox row. There is no dual write between the payment store and the ledger.
- Balances are a projection maintained asynchronously by the Balance Projector, which consumes posting events from the outbox and applies them in per-account micro-batches (at most one balance update per account per second). No payment-path transaction updates a balance row, which avoids hot-row contention on high-volume accounts such as fee_revenue or a large marketplace's merchant_payable. Projection lag is exported as a metric, and the Payout Service reads a balance only after the projector has passed the payout cut-off.
- The nightly recomputation from postings must equal the projection exactly; any difference pages the on-call engineer and freezes payouts for the affected merchant accounts.
- The ledger holds identifiers and amounts only — no customer personal data — so data-subject deletion requests never require modifying it.
- Authorization holds are recorded as memo entries in a separate memo ledger, not in the financial ledger, because no money moves at authorization.

15. Amounts, Currencies and FX

- Every amount is a signed 64-bit integer of minor units, paired with an ISO 4217 alphabetic currency code. The exponent for each currency comes from a versioned ISO 4217 reference table (SGD, MYR, IDR, THB, PHP, USD: 2; VND, JPY, KRW: 0).
- Where a provider expects a different exponent than ISO 4217 (for example, IDR as whole rupiah), the adapter converts explicitly and rejects any amount that would need rounding.
- A payment has a presentment currency (what the customer pays) and a settlement currency (what the merchant receives). Where they differ, the FX rate is quoted and locked at authorization, the quote ID is stored on the payment, and the FX conversion is booked as a separate journal through fx_position accounts at capture.
- Fee calculations use arbitrary-precision decimals inside the fee engine and round once, half-to-even, when the fee journal is created; the residue is tracked in a dedicated account.
- No floating-point type is used for amounts in any service, schema, or API payload. JSON amounts are integers; the API rejects decimals.

16. Reconciliation, Settlement and Payouts

16.1 Settlement file intake

Provider	Format	Delivery	Local delivery time
ACQ-SG1	CSV over SFTP	Daily	02:00 SGT
ACQ-SG2	ISO 20022 camt.053 + scheme-level CSV	Daily	03:00 SGT
ACQ-MY1	Fixed-width over SFTP	Daily	03:30 MYT
ACQ-ID1	CSV over SFTP	Daily	05:00 WIB
ACQ-TH1	CSV over SFTP	Daily	06:30 ICT
ACQ-PH1	XLSX over SFTP	Daily	04:00 PHT
WAG-1	JSON over S3 cross-account	Daily	04:00 SGT

Provider	Format	Delivery	Local delivery time
Sponsor banks (all rails)	camt.053	Daily	01:00–04:00 local

16.2 Matching

Files land in an S3 intake bucket; a parser per format normalises lines into `settlement_line` records. The batch matcher starts once all expected files for business day T have been received, so that cross-provider netting and multi-provider payouts are computed on a complete day. It performs a three-way match:

1. **Line** ↔ **payment attempt** on provider reference (acquirer's transaction ID or RRN), falling back to (amount, currency, card last 4, authorization code, date) when the reference is missing.
2. **Line** ↔ **ledger**: the matched attempt's capture journal must exist and its amount must equal the line's gross amount.
3. **Fees**: the line's fee must equal the contracted fee within a tolerance of one minor unit per line.

Matched payments transition to SETTLED and a settlement journal is posted. Unmatched and mismatched lines become reconciliation exceptions with a reason code and are routed to the Finance Operations queue. At 2025 volume, the matcher's measured end-to-end run time is 45 minutes, and merchant settlement report generation takes a further 15 minutes.

16.3 Payouts

The Payout Service computes each merchant account's payable balance after settlement, subtracts reserves and pending dispute holds, and instructs the sponsor bank by ISO 20022 pain.001 file or API. Payout instructions are idempotent on (merchant_account_id, payout date). A payout cannot be released while the merchant account has an open ledger-projection discrepancy (Section 14.3).

17. Merchant Webhooks

- **Events.** One event per state change, with a globally unique `event_id`, type (for example `payment.captured`), `created_at`, and a snapshot of the object including its `version`. Merchants deduplicate on `event_id` and order by object version; delivery order is not guaranteed.
- **Signing.** Each endpoint has its own signing secret, distinct from API keys and shown once at creation. The signature header carries a timestamp and `HMAC-SHA256(secret, timestamp + "." + raw_body)`. Merchant libraries reject signatures older than five minutes. Secrets can be rolled with a 24-hour overlap during which both are valid.
- **Delivery.** At-least-once. A delivery succeeds on any 2xx within 10 seconds. Failures are retried with jittered exponential backoff at approximately 1 min, 5 min, 15 min, 1 h, 3 h, 6 h, then every 12 h, up to 72 hours from first attempt (12 attempts including the first). After that the event is moved to a per-merchant dead-letter store, visible in the admin plane and replayable by API.
- **Isolation.** Each endpoint has a cap of 20 in-flight deliveries and its own circuit breaker, so a failing endpoint cannot starve other merchants.
- **Endpoint safety.** Endpoints must be HTTPS on port 443. The dispatcher resolves the hostname once at send time, refuses private, loopback, link-local and cloud-metadata addresses, and connects to the validated address for that delivery, which prevents server-side request forgery and DNS-rebinding against internal services.
- **Recovery.** Merchants can list and replay events for the past 30 days.

18. Merchant Admin Plane

18.1 Components

The admin plane consists of the Merchant Portal (web) and the Admin API (same capabilities, for merchants who automate operations). Both authenticate merchant users through the Serindit Pay identity service (email and password, with optional TOTP), and both authorise every call against the user's role.

18.2 Roles

Role	Permissions
Owner	All permissions; manage users and roles
Admin	Manage API keys, webhook endpoints, users (except Owner)
Finance	Issue refunds; view and export reports; edit payout bank account
Developer	View API keys (prefix only), manage webhook endpoints, view logs
Support	View payments (masked), issue refunds up to a per-refund limit set by the Owner
Viewer	Read-only dashboards

18.3 Authentication policy

- Passwords: minimum 12 characters, checked against a breached-password list, bcrypt (cost 12).
- TOTP multi-factor authentication is available to every user and is enforced for the Owner role at first login. Other roles may enable it in their profile.
- Sessions: 12-hour absolute, 30-minute idle timeout; login lockout after 10 failures in 15 minutes.

18.4 Payout bank account changes

A Finance or Owner user can change the payout bank account in the portal. The change takes effect from the next payout cycle. A confirmation email is sent to the user who made the change, and the change is written to the audit log.

18.5 Back-office console

Internal staff use a separate console behind corporate SSO with hardware-key MFA. Access is read-only to masked data by default; write actions (manual refunds, payout holds, merchant suspension) require just-in-time elevation approved by a second staff member, with a mandatory audit reason.

19. Data Model

Core tables in Aurora PostgreSQL (schema payments). Vault and settlement_line tables are specified in their component documents.

```
CREATE TABLE payment (
  payment_id          TEXT PRIMARY KEY,          -- pay_<ulid>
  merchant_account_id TEXT NOT NULL,
  sub_merchant_id     TEXT,
  state               TEXT NOT NULL,
  version              INTEGER NOT NULL DEFAULT 0,
  amount_minor        BIGINT NOT NULL CHECK (amount_minor > 0),
  currency             CHAR(3) NOT NULL,
  captured_minor       BIGINT NOT NULL DEFAULT 0,
  refunded_minor       BIGINT NOT NULL DEFAULT 0,
  settlement_currency  CHAR(3) NOT NULL,
  fx_quote_id         TEXT,
  payment_method_type  TEXT NOT NULL,          -- CARD | EWALLET | BANK_TRANSFER
  card_token          TEXT,                   -- ctk..., never PAN
```

```

card_bin8          CHAR(8),
card_last4         CHAR(4),
fraud_decision     TEXT,
fraud_score        NUMERIC(5,2),
merchant_reference TEXT,
created_at         TIMESTAMPTZ NOT NULL,
updated_at         TIMESTAMPTZ NOT NULL
);

```

```

CREATE TABLE payment_attempt (
    attempt_id      TEXT PRIMARY KEY,
    payment_id      TEXT NOT NULL REFERENCES payment(payment_id),
    seq             SMALLINT NOT NULL,
    provider_id     TEXT NOT NULL,
    outcome_class   TEXT,                -- APPROVED | FINAL | RETRYABLE
    provider_reference TEXT,
    response_code   TEXT,
    auth_code       TEXT,
    submitted_at    TIMESTAMPTZ NOT NULL,
    completed_at    TIMESTAMPTZ,
    UNIQUE (payment_id, seq)
);

```

```

CREATE TABLE journal (
    journal_id      TEXT PRIMARY KEY,
    payment_id      TEXT,
    journal_type     TEXT NOT NULL,
    reverses_journal_id TEXT REFERENCES journal(journal_id),
    reason_code      TEXT,
    approved_by     TEXT,
    created_at       TIMESTAMPTZ NOT NULL
);

```

```

CREATE TABLE posting (

    posting_id          BIGSERIAL PRIMARY KEY,

    journal_id          TEXT NOT NULL REFERENCES journal(journal_id),

    account_id          TEXT NOT NULL,

    direction           CHAR(1) NOT NULL CHECK (direction IN ('D','C')),

    amount_minor        BIGINT NOT NULL CHECK (amount_minor > 0),

    currency            CHAR(3) NOT NULL

);

CREATE TABLE outbox (

    outbox_id           BIGSERIAL PRIMARY KEY,

    aggregate_id        TEXT NOT NULL,

    event_type          TEXT NOT NULL,

    payload             JSONB NOT NULL,

    created_at          TIMESTAMPTZ NOT NULL,

    published_at        TIMESTAMPTZ

);

```

journal and posting have UPDATE and DELETE revoked from every application role; the migration role is the only role with DDL rights and is used only by the deployment pipeline.

19.1 Data placement

All primary data stores (Aurora, DynamoDB, ElastiCache, MSK, S3 intake and archive buckets) are in ap-southeast-1. The Aurora Global Database secondary in ap-southeast-3 (Jakarta) is used for disaster recovery only (Section 20). Retention: payment and attempt records 7 years (financial records); idempotency records 24 hours; webhook events 30 days; raw settlement files 7 years in S3 Glacier.

20. Resilience, Availability and Disaster Recovery

20.1 Within-region

- Every service runs at least three replicas spread across three AZs.
- Aurora PostgreSQL: writer plus two readers across three AZs; Aurora storage keeps six copies across three AZs; writer failover to a reader typically completes in under 60 seconds.
- ElastiCache for Redis: cluster mode, Multi-AZ with automatic failover.
- MSK: three brokers across three AZs, replication factor 3, `min.insync.replicas 2`, `producer acks=all`.
- DynamoDB: regional service, replicated across three AZs by AWS.

20.2 Cross-region

- Aurora Global Database replicates the payments and ledger cluster to ap-southeast-3. Replication is storage-level and asynchronous, with typical lag under one second.
- MSK, DynamoDB and ElastiCache are not replicated cross-region; on regional failover they are recreated

empty in ap-southeast-3 from infrastructure-as-code, and in-flight idempotency state is reconstructed from the payment table.

- Regional failover is a runbook-driven managed operation (promote the Aurora secondary, scale up the standby EKS cluster, switch Route 53 records), rehearsed twice a year. Target duration: 30 minutes.

20.3 Stuck-payment sweeper

Every 60 seconds, the Orchestrator's sweeper finds payments in AUTHORISING for more than 30 seconds, queries the provider's status endpoint where one exists, and resolves the payment to AUTHORISED or FAILED. Payments still unresolved after 10 minutes are escalated to the operations queue. The sweeper also completes idempotency records left IN_PROGRESS.

20.4 Dependency degradation

Dependency failure	Behaviour
One acquirer down	Circuit breaker opens; traffic routes to remaining eligible acquirers
Fraud vendor down	Default decisions per risk tier (Section 13.2)
Redis down	Fast path skipped; DynamoDB path serves all idempotency checks
MSK down	Outbox accumulates in Aurora; relay drains on recovery; webhooks delayed, payments unaffected

21. Latency and Capacity Budget

21.1 Card authorization latency budget (p99)

Step	p99 (ms)	Basis
Edge, TLS, API key authentication	25	Load-test spike
Rate limit + validation	3	In-process
Idempotency (Redis miss + DynamoDB conditional put)	12	DynamoDB single-digit-ms writes
Fraud Hook	150	Hard timeout
Routing decision	5	In-memory rules and priors
Vault detokenise (HSM unwrap)	20	Vendor benchmark
Acquirer round trip	1,100	2025 production p99, slowest acquirer
State transition + ledger + outbox commit	15	Aurora, same AZ writer
Idempotency completion + response	8	
Total	1,338	

Summing component p99s gives a conservative 1,338 ms against the 1,500 ms target in NFR-2. Cascading does not move the p99 because fewer than 1% of card payments cascade; the cascade path therefore sits above the 99th percentile. Orchestration overhead (everything except the acquirer round trip) is 238 ms against the 250 ms target.

21.2 Capacity

- **Payments API and Orchestrator.** Stateless; pre-scaled for 2,500 TPS before known campaign events.
- **Aurora.** Each card payment produces roughly nine row writes (payment, attempt, two state updates, journal, two to three postings, outbox). At 2,000 TPS that is about 18,000 row writes per second. A spike on `db.r7g.8xlarge` sustained 2,600 TPS of the full write mix for two hours at 58% writer CPU with commit latency p99 of 11 ms. Production uses `db.r7g.12xlarge` for headroom.
- **MSK.** About six 1.8 KB events per payment; 2,000 TPS $\approx$ 21.6 MB/s ingress across three `kafka.m7g.xlarge` brokers.

22. Observability and Audit

- **Tracing.** OpenTelemetry with `payment_id` and `merchant_id` as span attributes; all payment events are also

written to the event bus, which is the basis for NFR-11.

- **Logging.** A shared logging library drops deny-listed fields (card_number, cvc, password, secret, authorization header) and Luhn-scrubs free text; CDE logs never leave the CDE account.
- **Audit.** Audit records (actor, action, target, before/after, request ID, source IP, timestamp) go to an append-only table and to S3 with compliance-mode Object Lock for five years (NFR-12).

23. Prior Art and Reference Architecture

MPOP is deliberately conventional in its core and borrows from published practice.

Reference	What it does	What we take	Why not adopt wholesale
Juspay Hyperswitch (open source)	Full orchestrator: connectors, routing, retries, vault	Connector interface shape; outcome taxonomy; routing rule concepts	Thinner coverage of SEA wallets and rails than our integrations; ledger and reconciliation needs exceed its scope; we would still own its PCI posture.
Commercial orchestrators (Spredly, Primer, Gr4vy)	Hosted vault and routing	Vault-first PCI scoping pattern	Per-transaction pricing compared unfavourably with build cost in the Conceptual Design's five-year cost model; acquirer and local-rail coverage gaps in ID, TH and PH.
Stripe API documentation; IETF HTTPAPI draft "The Idempotency-Key HTTP Header Field"	Idempotency keys, webhook signing, object versioning	Idempotency-Key semantics (24-hour retention; 409 on an in-flight duplicate and 422 on payload mismatch, per the IETF draft); timestamped HMAC webhook signatures	— (design pattern, not a product)
Modern Treasury / Formance ledger designs	Double-entry ledgers for payments companies	Journal/posting model; append-only with reversing entries; balance projections verified against postings	We need the ledger in the same transaction as payment state, which rules out an external ledger service.
Transactional outbox (Richardson, <i>Microservices Patterns</i>)	Atomic state change plus event	Outbox relayed to Kafka	—

24. Confirmed Decisions

Decision	Answer
Cloud and primary region	AWS, ap-southeast-1 (Singapore), three AZs
Compute	Amazon EKS; CDE on a separate EKS cluster in a separate AWS account
Payments and ledger store	Aurora PostgreSQL 16, db.r7g.12xlarge, writer + 2 readers
Disaster recovery	Aurora Global Database secondary in ap-southeast-3; runbook failover, 30-minute target
Event bus	Amazon MSK, 3 brokers, RF 3
Idempotency store	Redis fast path (idem:resp:{key}) + DynamoDB payments-idempotency (PK merchant_id, SK idempotency_key, LSI on created_at)
Idempotency retention	24 hours
Vault	In-house, envelope encryption, KEKs in AWS CloudHSM (one HSM at launch)
CVC handling	Encrypted in Vault until settlement, max 72 hours
Card-on-file credential	Network tokens (VTS/MDES) as the primary credential for all card-on-file and recurring payments from launch; PAN fallback only where issuer unsupported
Cascade policy	Up to 2 additional attempts; triggers: soft declines 05/91/96, 5xx, connection refused, 2,500 ms timeout
Routing	Weighted cost/approval/latency score; cost-preferred within 2 tolerance
Fraud	FRV-1, synchronous, 150 ms timeout, risk-tier fallback

Decision	Answer
Ledger	Double-entry, append-only, same transaction as state change, outbox
Money representation	Signed 64-bit integer minor units + ISO 4217 code
Reconciliation	Daily batch after all files received; three-way match
Webhooks	At-least-once, HMAC-SHA256 timestamped signatures, 72-hour bounded retry, DLQ + replay
Merchant user MFA	TOTP; enforced for Owner, optional for other roles
Back-office access	Corporate SSO + hardware MFA; JIT elevation with second approver

25. Pending Backlog

Items confirmed as in scope but not yet fully designed:

1. **Dispute and chargeback module** — case management, evidence submission, representment deadlines per scheme. Disputes are ingested as ledger events in v1.
2. **Token Requestor onboarding** — Visa VTS and Mastercard MDES token-requestor registration (TRID) and the commercial agreement with a token service provider; certification test plan. Application not yet submitted.
3. **Smart-routing model** — replace static 28-day approval priors with a per-transaction approval-probability model; requires six months of attempt-level data from MPOP.
4. **FRV-1 contract** — data processing agreement and SLA finalisation.
5. **ACQ-PH1 API v3** — adapter migration before the v2 retirement in Q3 2027.
6. **Multi-currency payouts and marketplace split rules** — beyond a single settlement currency and fixed commission.
7. **Admin API request/response schemas** — endpoint list is fixed; JSON Schemas and error catalogue to be written.

26. Validation and Acceptance Criteria

Each requirement from Section 2 is validated by a specific method with a concrete pass/fail acceptance criterion. This table is the basis for the test plan in Build Phase 9.

26.1 Functional requirements

ID	Validation method	Acceptance criteria
FR-1	API contract test suite	Every endpoint returns the documented status and schema for valid and invalid requests, per method family.
FR-2	State machine property test	Randomised event sequences (10 ⁶ per run) never produce a transition outside Section 7.2; every acknowledged transition is present in the database after a forced pod kill.
FR-3	Connector conformance suite	Each adapter passes the shared suite against the provider sandbox.
FR-4	API negative test	Mutating requests without Idempotency-Key return 400; retrieve requests succeed without it.
FR-5	Idempotency replay test	Send a create-payment request; after its response is received, resend the identical request with the same Idempotency-Key. The second response is byte-identical to the first and the acquirer simulator records exactly one authorization. Resending with a modified body returns 422.
FR-6	Routing eligibility test	For a matrix of eligibility inputs, the eligible set equals the expected set.
FR-7	Cascade simulation	With the acquirer simulator injecting each retryable and final outcome, cascades occur only on retryable outcomes, never exceed two additional attempts, and stop at scheme reattempt limits.
FR-8	Routing decision test	For a set of synthetic transactions with configured costs and approval priors, the selected acquirer matches the expected choice.
FR-9	Fraud hook test	ACCEPT, REVIEW, REJECT and timeout each produce the documented outcome.
FR-10	Ledger invariant test	After a 1-million-payment simulation including captures, refunds, settlements and payouts, every journal balances per currency and the projection equals the recomputation.
FR-11	Reconciliation golden-file test	For each provider's sample file with seeded mismatches, every seeded mismatch becomes an exception with the correct reason code and every other line matches.
FR-12	Webhook delivery test	Every state change emits exactly one event; signatures verify with the reference library; a failing endpoint

ID	Validation method	Acceptance criteria
		receives the documented retry schedule and the event lands in the DLQ after 72 hours.
FR-13	Admin plane role matrix test	Each role can perform exactly the actions in Section 18.2.
FR-14	Stored-credential test	A card-on-file payment uses a network token and cryptogram for a network-token-enabled test card, and correct MIT/CIT indicators.
FR-15	Refund test per method	Full and partial refunds succeed per method; payout-based refunds produce a payout instruction.
FR-16	Audit completeness test	Scripted mutations and admin actions each produce exactly one complete audit record.
FR-17	Payout schedule test	Each schedule produces payout instructions on the correct dates over a simulated month.

26.2 Non-functional requirements

ID	Validation method	Acceptance criteria
NFR-1	Load test	2,000 TPS for four hours against acquirer simulators with production latency distributions; error rate below 0.1%; NFR-2 met throughout.
NFR-2	Latency benchmark	Under the NFR-1 load with 3.8% retryable-outcome injection, p99 end-to-end card authorization $\leq$ 1,500 ms and orchestration overhead p99 $\leq$ 250 ms.
NFR-3	Availability SLO tracking	Synthetic transactions every 10 seconds from three external locations; monthly success ratio $\geq$ 99.95%.
NFR-4	Regional DR game day	Unplanned failover of the Aurora Global Database to ap-southeast-3; every ledger posting committed in ap-southeast-1 before the failure is present after failover; service restored within 30 minutes.
NFR-5	PCI DSS assessment	Report on Compliance by a QSA with CDE as defined in Section 12.2; segmentation test passes.
NFR-6	Vault SAD test	CVC records are encrypted at rest and absent after the associated payment settles or after 72 hours.
NFR-7	Privacy review	DPO sign-off per market; every personal data field is mapped to a purpose and retention period.
NFR-8	Reconciliation SLA test	On a production-volume replay, auto-match $\geq$ 99.9% and reports published by 08:00 SGT.
NFR-9	Noisy-neighbour test	One merchant at 3 $\times$ its rate limit does not move other merchants' p99 by more than 5%.
NFR-10	Connector onboarding dry run	A sandbox acquirer is onboarded with changes only to its adapter and configuration.
NFR-11	Trace completeness test	For 1,000 sampled payments, every step is retrievable by payment_id within 5 s.
NFR-12	Audit immutability test	Modifying or deleting an audit object in S3 fails under Object Lock.

27. Implementation Readiness Assessment

Could an engineering team build each component from this document without further design decisions?

Component	Ready?	What remains to decide
Payment state machine and data model	Ready	Sections 7 and 19.
Payments API (merchant-facing)	Ready	Operations and idempotency contract specified (Sections 7, 9).
Idempotency Layer	Ready	Two-tier design, key schema and capacity specified (Section 9).
Routing Engine	Mostly ready	Eligibility and ranking specified; approval priors static until the model in Backlog item 3.
Retry Engine	Ready	Outcome classes, cascade limits and scheme rules specified (Section 11).
Card Vault and Card Adapter	Ready	Tokenisation, envelope encryption, HSM, SAD handling and network tokens specified (Section 12).
Fraud Hook	Mostly ready	Payload and fallbacks specified; vendor contract pending (Backlog item 4).
Ledger	Ready	Accounts, journals, invariants and examples specified (Section 14).
Reconciliation	Mostly ready	Matching rules specified; per-provider parsers require sample files from ACQ-PH1 and WAG-1.

Component	Ready?	What remains to decide
Payouts	Ready	Section 16.3.
Webhooks	Ready	Section 17.
Admin plane	Partial	Roles and policies specified; Admin API schemas pending (Backlog item 7).
Disputes	Not ready	Backlog item 1.

Overall conclusion

The transaction core is specified to implementation level. The Admin API schemas and the dispute module are the two material gaps; neither blocks the first merchant cohort, which will use the portal and have disputes handled manually by Finance Operations.

28. Build Phases

#	Phase	Depends on open design items?
1	Foundations — AWS accounts, EKS, Aurora, MSK, DynamoDB, IaC, CI/CD, observability baseline	No — ready
2	CDE — Vault, CloudHSM, Card Adapter (ACQ-SG1, ACQ-SG2), PCI segmentation	No — ready
3	Transaction core — Payments API, idempotency, state machine, ledger, outbox; card-on-file with network tokens	No — ready
4	Routing and Retry Engines; remaining card adapters	Partially — static priors until Backlog item 3
5	Wallet and bank-transfer adapters (WAG-1, PayNow, FPX, DuitNow, QRIS, PromptPay, InstaPay)	No — ready
6	Reconciliation and payouts	Partially — sample files outstanding
7	Webhooks and merchant admin plane	Partially — Admin API schemas (Backlog item 7)
8	Fraud Hook (FRV-1)	Yes — vendor contract (Backlog item 4)
9	Hardening — load, chaos and DR game days; PCI DSS assessment; Section 26 test plan	Follows Section 26

The first merchant cohort (Singapore, cards and PayNow) goes live after Phase 7; other markets follow at four-week intervals. No implementation has begun.